

Data Mining (DS3002)

Assignment 4: Heartbeat to Heatmap

Student Name: Abdullah Tariq

Roll Number: 23i-2091

****Student:** Abdullah Tariq ****Roll Number:** 23i-2091****

This notebook follows the instructions from the assignment document ‘DS3002 Assignment4 DataMining.docx’. It covers data loading, preprocessing, model training (XGBoost), model interpretation (SHAP), and visualization (t-SNE).

```
1 # Install required packages (run once)
2 # Note: This cell was previously interrupted. If you encounter ModuleNotFoundError
   errors,
3 # please run this cell again to ensure all dependencies are installed.
4 !pip install -q python-docx pandas scikit-learn xgboost shap matplotlib seaborn
   tqdm
```

Listing 1: Code Implementation

```
1 from docx import Document
2 import pathlib, json, os
3 doc_path = pathlib.Path('DS3002 Assignment4 DataMining.docx')
4 if doc_path.exists():
5     doc = Document(doc_path)
6     full_text = '\n'.join([para.text for para in doc.paragraphs])
7     print('--- Assignment Description ---')
8     print(full_text[:1000]) # preview first 1000 chars
9 else:
10    print('Assignment docx not found')
```

Listing 2: Code Implementation

Output

Assignment docx not found

1 Load the dataset The dataset ‘processed.cleveland.data’ is a cleaned version of the classic Cleveland Heart Disease dataset. It contains 14 attributes (13 features + target).

```
1 import pandas as pd
2 import pathlib
3 data_path = pathlib.Path('processed.cleveland.data')
4 col_names = [
5     'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
6     'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
7 ]
8 df = pd.read_csv(data_path, header=None, names=col_names)
9 df.head()
```

Listing 3: Code Implementation

2 Pre-processing * Replace missing values (encoded as ‘?’) with ‘NaN’ and impute with column median. * Convert the target to binary: 0 = no disease, 1 = disease (original values 0-4).

```
1 import numpy as np
2 df.replace('?', np.nan, inplace=True)
3 # Convert numeric columns to float
4 for col in df.columns:
5     df[col] = pd.to_numeric(df[col])
6 # Impute missing values with median
7 df.fillna(df.median(), inplace=True)
```

```

8 # Binary target
9 df['target'] = (df['target'] > 0).astype(int)
10 df.head()

```

Listing 4: Code Implementation

3 Train-test split

```

1 from sklearn.model_selection import train_test_split
2 X = df.drop('target', axis=1)
3 y = df['target']
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42, stratify=y)
6 print('Train shape:', X_train.shape, 'Test shape:', X_test.shape)

```

Listing 5: Code Implementation

Output

```
Train shape: (242, 13) Test shape: (61, 13)
```

4 Model – XGBoost Classifier

```

1 from xgboost import XGBClassifier
2 model = XGBClassifier(
3     n_estimators=200,
4     learning_rate=0.05,
5     max_depth=4,
6     subsample=0.8,
7     colsample_bytree=0.8,
8     random_state=42,
9     use_label_encoder=False,
10    eval_metric='logloss'
11 )
12 model.fit(X_train, y_train)
13 # Save model for later reuse
14 import joblib
15 joblib.dump(model, 'xgb_model.joblib')

```

Listing 6: Code Implementation

Output

```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [18:20:00] WARNING:
Parameters: { "use_label_encoder" } are not used.

```

```
bst.update(dtrain, iteration=i, fobj=obj)
```

5 Evaluation We compute accuracy, confusion matrix, classification report and plot the ROC curve.

```

1 from sklearn.metrics import accuracy_score, confusion_matrix,
2   classification_report, roc_auc_score, roc_curve
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 y_pred = model.predict(X_test)

```

```

5 y_proba = model.predict_proba(X_test)[: ,1]
6 print('Accuracy:', accuracy_score(y_test, y_pred))
7 print('ROC AUC:', roc_auc_score(y_test, y_proba))
8 print('Classification Report:', classification_report(y_test, y_pred))
9 cm = confusion_matrix(y_test, y_pred)
10 plt.figure(figsize=(5,4))
11 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
12 plt.xlabel('Predicted')
13 plt.ylabel('Actual')
14 plt.title('Confusion Matrix')
15 plt.show()

```

Listing 7: Code Implementation

Output

```

Accuracy: 0.8852459016393442
ROC AUC: 0.9404761904761905
Classification Report:

```

		precision	recall	f1-score	support
	0	0.96	0.82	0.89	33
	1	0.82	0.96	0.89	28
	accuracy			0.89	61
	macro avg	0.89	0.89	0.89	61
	weighted avg	0.90	0.89	0.89	61

6 Model Interpretation with SHAP

```

1 import shap
2 # Explain the model (use a sample of the training data for speed)
3 explainer = shap.TreeExplainer(model)
4 shap_values = explainer.shap_values(X_train)
5 # Summary plot
6 shap.summary_plot(shap_values, X_train, plot_type='bar')

```

Listing 8: Code Implementation

7 t-SNE visualization of the feature space

```

1 from sklearn.manifold import TSNE
2 import numpy as np
3 # Combine features and target for coloring
4 tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=1000)
5 X_embedded = tsne.fit_transform(X)
6 plt.figure(figsize=(8,6))
7 scatter = plt.scatter(X_embedded[:,0], X_embedded[:,1], c=y, cmap='viridis',
8                       alpha=0.7)
9 plt.colorbar(scatter, label='Target')
10 plt.title('t SNE of Heart Disease Data')
11 plt.xlabel('Component 1')
12 plt.ylabel('Component 2')
13 plt.show()

```

Listing 9: Code Implementation

Output

```
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter'
  warnings.warn(
```

Feel free to experiment with hyper-parameters, try other models (Logistic Regression, Random Forest), or add more visualizations.

Now, let's implement a simple Multi-Layer Perceptron (MLP) for the heart disease prediction task. This will cover the ANN/SLP/MLP requirement from the assignment.

```
1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense, Dropout
4
5 # Define the MLP model
6 mlp_model = Sequential([
7     Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
8     Dropout(0.3),
9     Dense(32, activation='relu'),
10    Dropout(0.3),
11    Dense(1, activation='sigmoid') # Output layer for binary classification
12 ])
13
14 # Compile the model
15 mlp_model.compile(
16     optimizer='adam',
17     loss='binary_crossentropy',
18     metrics=['accuracy']
19 )
20
21 mlp_model.summary()
```

Listing 10: Code Implementation

Output

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not p
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Train the MLP Model

```
1 history = mlp_model.fit(
2     X_train,
3     y_train,
4     epochs=50,
5     batch_size=32,
6     validation_split=0.2,
7     verbose=0 # Set to 1 for progress bars
8 )
9
10 print('MLP Model Training Complete.')
11
12 # Save MLP model
13 mlp_model.save('mlp_model.h5')
```

Listing 11: Code Implementation

Output

WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_m

Output

MLP Model Training Complete.

Evaluate the MLP Model

```

1 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report, roc_auc_score, roc_curve
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Predict probabilities and classes on the test set
6 y_pred_mlp_proba = mlp_model.predict(X_test)
7 y_pred_mlp = (y_pred_mlp_proba > 0.5).astype(int)
8
9 # Evaluate and print metrics
10 print('MLP Accuracy:', accuracy_score(y_test, y_pred_mlp))
11 print('MLP ROC AUC:', roc_auc_score(y_test, y_pred_mlp_proba))
12 print('MLP Classification Report:\n', classification_report(y_test, y_pred_mlp)
13       )
14
15 # Plot Confusion Matrix
16 cm_mlp = confusion_matrix(y_test, y_pred_mlp)
17 plt.figure(figsize=(5,4))
18 sns.heatmap(cm_mlp, annot=True, fmt='d', cmap='Blues', cbar=False)
19 plt.xlabel('Predicted')
20 plt.ylabel('Actual')
21 plt.title('MLP Confusion Matrix')
22 plt.show()

```

Listing 12: Code Implementation

Output

```

[1m2/2[0m [32m[0m[37m[0m [1m0s[0m 89ms/step
MLP Accuracy: 0.6557377049180327
MLP ROC AUC: 0.7683982683982684
MLP Classification Report:

```

	precision	recall	f1-score	support
0	0.66	0.76	0.70	33
1	0.65	0.54	0.59	28
accuracy			0.66	61
macro avg	0.66	0.65	0.65	61
weighted avg	0.66	0.66	0.65	61

Let's briefly compare the performance of the two models trained.

```

1 print(f"XGBoost Accuracy: {accuracy_score(y_test, y_pred):.4f}")
2 print(f"MLP Accuracy: {accuracy_score(y_test, y_pred_mlp):.4f}")
3 print(f"XGBoost ROC AUC: {roc_auc_score(y_test, y_proba):.4f}")

```

```
4 print(f"MLP ROC AUC: {roc_auc_score(y_test, y_pred_mlp_proba):.4f}")
```

Listing 13: Code Implementation

Output

```
XGBoost Accuracy: 0.8852
MLP Accuracy: 0.6557
XGBoost ROC AUC: 0.9405
MLP ROC AUC: 0.7684
```

To address the digit recognition requirement, we will use the MNIST dataset, a classic dataset of handwritten digits. This involves loading the dataset, normalizing pixel values, reshaping the images for CNN input, and one-hot encoding the labels.

```
1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3 from tensorflow.keras.utils import to_categorical
4
5 # Load MNIST dataset
6 (X_train_mnist, y_train_mnist), (X_test_mnist, y_test_mnist) = mnist.load_data
7   ()
8
9 # Preprocess images
10 # Reshape to be [samples][width][height][channels]
11 X_train_mnist = X_train_mnist.reshape(X_train_mnist.shape[0], 28, 28, 1).astype
12   ('float32')
13 X_test_mnist = X_test_mnist.reshape(X_test_mnist.shape[0], 28, 28, 1).astype('
14   float32')
15
16 # Normalize pixel values to 0-1
17 X_train_mnist = X_train_mnist / 255
18 X_test_mnist = X_test_mnist / 255
19
20 # One-hot encode target labels
21 y_train_mnist = to_categorical(y_train_mnist)
22 y_test_mnist = to_categorical(y_test_mnist)
23
24 print('MNIST Training data shape:', X_train_mnist.shape, y_train_mnist.shape)
25 print('MNIST Test data shape:', X_test_mnist.shape, y_test_mnist.shape)
```

Listing 14: Code Implementation

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
[1m11490434/11490434[0m [32m[0m[37m[0m [1m0s[0m 0us/step
MNIST Training data shape: (60000, 28, 28, 1) (60000, 10)
MNIST Test data shape: (10000, 28, 28, 1) (10000, 10)
```

We will define a simple CNN model suitable for image classification tasks like MNIST. This model will consist of convolutional layers, pooling layers, and dense layers for classification.

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
3   Dropout
4
5 # Define the CNN model
```



```

5 cnn_model = Sequential([
6     Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
7     MaxPooling2D((2, 2)),
8     Conv2D(64, (3, 3), activation='relu'),
9     MaxPooling2D((2, 2)),
10    Flatten(),
11    Dense(128, activation='relu'),
12    Dropout(0.5),
13    Dense(10, activation='softmax') # 10 classes for digits 0-9
14 ])
15
16 # Compile the model
17 cnn_model.compile(
18     optimizer='adam',
19     loss='categorical_crossentropy',
20     metrics=['accuracy']
21 )
22
23 cnn_model.summary()

```

Listing 15: Code Implementation

Output

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Train the CNN Model

```

1 cnn_history = cnn_model.fit(
2     X_train_mnist,
3     y_train_mnist,
4     epochs=10,
5     batch_size=128,
6     validation_split=0.2,
7     verbose=0 # Set to 1 for progress bars
8 )
9
10 print('CNN Model Training Complete.')
11
12 # Save CNN model
13 cnn_model.save('cnn_mnist_model.h5')

```

Listing 16: Code Implementation

Output

```

WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_m

```

Output

```

CNN Model Training Complete.

```

Evaluate the CNN Model

```

1 loss_mnist, accuracy_mnist = cnn_model.evaluate(X_test_mnist, y_test_mnist,
2     verbose=0)
3 print(f'CNN Test Accuracy (MNIST): {accuracy_mnist:.4f}')
```

```
3 print(f'CNN Test Loss (MNIST): {loss_mnist:.4f}')
```

Listing 17: Code Implementation

Output

CNN Test Accuracy (MNIST): 0.9915

CNN Test Loss (MNIST): 0.0280

To fulfill the assignment's requirement for an interactive prediction application ('app.py') and the submission of a 'requirements.txt' file, we will provide the content for these files. These files would typically be run outside this Colab notebook environment, for instance, using a web framework like Flask or Streamlit.

The 'app.py' will include logic to load the trained models (XGBoost for heart disease, CNN for digit recognition) and provide an interface for predictions, incorporating the specific display requirements (color-coded risk, confidence, top 3 features, plain-English explanation). The 'requirements.txt' will list all necessary Python libraries. `### 'app.py' Content (Conceptual Structure)`

Below is a conceptual structure for 'app.py'. You would need to adapt this to a specific web framework (e.g., Flask, Streamlit) and implement the UI logic to capture input and display the results as specified in the assignment. The SHAP explanation part for individual predictions can be generated using 'shap.Explainer' and 'shap.explain_instance' on new input data.

```
1 app_py_content = '''
2 import streamlit as st
3 import pandas as pd
4 import numpy as np
5 import joblib
6 import tensorflow as tf
7 from tensorflow.keras.models import load_model
8 import shap
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 import io
12
13 # Load models
14 xgb_model = joblib.load('xgb_model.joblib')
15 mlp_model = load_model('mlp_model.h5')
16 cnn_model = load_model('cnn_mnist_model.h5')
17
18 # Load X_train for SHAP explainer (or use representative data)
19 X = pd.read_csv('processed.cleveland.data', header=None, names=[
20     'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
21     'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
22 ])
23 X.replace('?', np.nan, inplace=True)
24 for col in X.columns:
25     X[col] = pd.to_numeric(X[col])
26 X.fillna(X.median(), inplace=True)
27 X_train_shap = X.drop('target', axis=1)
28
29 # SHAP explainer for XGBoost
30 xgb_explainer = shap.TreeExplainer(xgb_model)
31
32 # Feature names for heart disease model
33 heart_disease_features = [
34     'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
35     'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
```

```

36 ]
37
38 st.set_page_config(layout='wide')
39 st.title("CardioAI Labs Decision Support Tool")
40
41 st.sidebar.header("Choose Module")
42 module_choice = st.sidebar.radio(
43     "Select a task:",
44     ('Heart Disease Prediction', 'Digit Recognition')
45 )
46
47 if module_choice == 'Heart Disease Prediction':
48     st.header("Heart Disease Prediction")
49     st.write("Enter patient data to predict heart disease risk.")
50
51     with st.form("heart_disease_form"):
52         age = st.number_input('Age', min_value=1, max_value=120, value=50)
53         sex = st.selectbox('Sex', options=[(0, 'Female'), (1, 'Male')],
54                             format_func=lambda x: x[1])[0]
55         cp = st.selectbox('Chest Pain Type (CP)', options=[(1, 'Typical Angina'),
56                                                         (2, 'Atypical Angina'), (3, 'Non-Anginal Pain'), (4, 'Asymptomatic')],
57                             format_func=lambda x: x[1])[0]
58         trestbps = st.number_input('Resting Blood Pressure (trestbps)',
59                                     min_value=80, max_value=200, value=120)
60         chol = st.number_input('Cholesterol (chol)', min_value=100, max_value=
61                                     =600, value=200)
62         fbs = st.selectbox('Fasting Blood Sugar > 120 mg/dl (fbs)', options
63                             =[(0, 'No'), (1, 'Yes')], format_func=lambda x: x[1])[0]
64         restecg = st.selectbox('Resting Electrocardiographic Results (restecg)
65                                 ', options=[(0, 'Normal'), (1, 'ST-T wave abnormality'), (2, 'Left
66                                 Ventricular Hypertrophy')], format_func=lambda x: x[1])[0]
67         thalach = st.number_input('Maximum Heart Rate Achieved (thalach)',
68                                     min_value=60, max_value=220, value=150)
69         exang = st.selectbox('Exercise Induced Angina (exang)', options=[(0, '
70                                 No'), (1, 'Yes')], format_func=lambda x: x[1])[0]
71         oldpeak = st.number_input('ST Depression Induced by Exercise Relative
72                                 to Rest (oldpeak)', min_value=0.0, max_value=6.0, value=1.0, step=0.1)
73         slope = st.selectbox('Slope of the Peak Exercise ST Segment (slope)',
74                                 options=[(1, 'Upsloping'), (2, 'Flat'), (3, 'Downsloping')], format_func=
75                                 lambda x: x[1])[0]
76         ca = st.selectbox('Number of Major Vessels Colored by Flouroscopy (ca)
77                                 ', options=[(0, '0'), (1, '1'), (2, '2'), (3, '3')], format_func=lambda x:
78                                 str(x[1]))[0]
79         thal = st.selectbox('Thallium Stress Test Result (thal)', options=[(3,
80                                 'Normal'), (6, 'Fixed Defect'), (7, 'Reversible Defect')], format_func=
81                                 lambda x: x[1])[0]
82
83         predict_button = st.form_submit_button("Predict Heart Disease")
84
85     if predict_button:
86         input_data = pd.DataFrame([[
87             age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak
88             , slope, ca, thal
89         ]], columns=heart_disease_features)
90
91         # XGBoost Prediction
92         xgb_proba = xgb_model.predict_proba(input_data)[: , 1][0]
93         xgb_pred = (xgb_proba > 0.5).astype(int)
94
95         # Display prediction with color-coded risk label
96         risk_label = "Disease Present" if xgb_pred == 1 else "No Disease"
97         risk_color = "red" if xgb_pred == 1 else "green"
98         st.subheader(f"Prediction: :{risk_color} [{risk_label}]")

```

```

81
82     # Model's confidence score
83     st.write(f"Confidence: **{xgb_proba*100:.2f}%**")
84
85     # SHAP explanation for top 3 features
86     shap_values_instance = xgb_explainer.shap_values(input_data)
87     feature_impact = pd.Series(shap_values_instance[0], index=
heart_disease_features)
88     top_3_features = feature_impact.abs().sort_values(ascending=False).head
(3)
89
90     st.write("**Top 3 Features Driving Prediction:**")
91     fig, ax = plt.subplots(figsize=(8, 4))
92     top_3_features.plot(kind='barh', ax=ax, color='skyblue')
93     ax.set_title('Feature Importance')
94     ax.set_xlabel('SHAP Value (Magnitude)')
95     st.pyplot(fig)
96
97     # Plain-English explanation
98     st.write("**Explanation for Nurse:**")
99     explanation_text = "Based on the patient's data, "
100    # Add more specific explanations based on feature_impact, e.g.,
101    # if 'cp' is high and positive impact: "chest pain type was a strong
indicator..."
102    # This part requires more advanced logic to generate dynamically
103    st.write("This patient's health indicators have been analyzed by the
model. The primary factors influencing this prediction are [Feature 1], [
Feature 2], and [Feature 3]. Please review these with the patient.")
104
105
106 elif module_choice == 'Digit Recognition':
107     st.header("Digit Recognition")
108     st.write("Upload an image of a handwritten digit (28x28 grayscale) for
prediction.")
109
110     uploaded_file = st.file_uploader("Choose a digit image...", type=["png", "
jpg", "jpeg"])
111
112     if uploaded_file is not None:
113         from PIL import Image
114         image = Image.open(uploaded_file).convert('L') # Convert to grayscale
115         image = image.resize((28, 28))
116         img_array = np.array(image) / 255.0
117         img_array = img_array.reshape(1, 28, 28, 1)
118
119         st.image(image, caption='Uploaded Image', use_column_width=True)
120         st.write("")
121
122         if st.button("Predict Digit"):
123             predictions = cnn_model.predict(img_array)
124             predicted_digit = np.argmax(predictions)
125             confidence = np.max(predictions) * 100
126
127             st.subheader(f"Predicted Digit: **{predicted_digit}**")
128             st.write(f"Confidence: **{confidence:.2f}%**")
129     '''
130
131 # Save this content to a file (e.g., app.py) manually or programmatically
outside the notebook
132 # with open('app.py', 'w') as f:
133 #     f.write(app_py_content)
134 # print("Content for app.py has been generated.")
135

```

```
136 # For demonstration purposes, print the content
137 print("--- Content for app.py ---")
138 print(app_py_content)
139 print("\n--- End of app.py Content ---")
```

Listing 18: Code Implementation

Output

```
--- Content for app.py ---
```

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
import tensorflow as tf
from tensorflow.keras.models import load_model
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import io

# Load models
xgb_model = joblib.load('xgb_model.joblib')
mlp_model = load_model('mlp_model.h5')
cnn_model = load_model('cnn_mnist_model.h5')

# Load X_train for SHAP explainer (or use representative data)
X = pd.read_csv('processed.cleveland.data', header=None, names=[
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
])
X.replace('?', np.nan, inplace=True)
for col in X.columns:
    X[col] = pd.to_numeric(X[col])
X.fillna(X.median(), inplace=True)
X_train_shap = X.drop('target', axis=1)

# SHAP explainer for XGBoost
xgb_explainer = shap.TreeExplainer(xgb_model)

# Feature names for heart disease model
heart_disease_features = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
]

st.set_page_config(layout='wide')
st.title("CardioAI Labs Decision Support Tool")

st.sidebar.header("Choose Module")
module_choice = st.sidebar.radio(
    "Select a task:",
    ('Heart Disease Prediction', 'Digit Recognition')
)

if module_choice == 'Heart Disease Prediction':
    st.header("Heart Disease Prediction")
    st.write("Enter patient data to predict heart disease risk.")

    with st.form("heart_disease_form"):
        age = st.number_input('Age', min_value=1, max_value=120, value=50)
        sex = st.selectbox('Sex', options=[(0, 'Female'), (1, 'Male')], format_func=lambda x: x[1])
        cp = st.selectbox('Chest Pain Type (CP)', options=[(1, 'Typical Angina'), (2, 'Atypical A
        trestbps = st.number_input('Resting Blood Pressure (trestbps)', min_value=80, max_value=2
        chol = st.number_input('Cholesterol (chol)', min_value=100, max_value=600, value=200)
        fbs = st.selectbox('Fasting Blood Sugar > 120 mg/dl (fbs)', options=[(0, 'No'), (1, 'Yes')
        restecg = st.selectbox('Resting Electrocardiographic Results (restecg)', options=[(0, 'No
        thalach = st.number_input('Maximum Heart Rate Achieved (thalach)', min_value=60, max_valu
        exang = st.selectbox('Exercise Induced Angina (exang)', options=[(0, 'No'), (1, 'Yes')],
        oldpeak = st.number_input('ST Depression Induced by Exercise Relative to Rest (oldpeak)',
        slope = st.selectbox('Slope of the Peak Exercise ST Segment (slope)', options=[(1, 'Upward
```

'requirements.txt' Content

This file lists all the Python packages required to run the 'app.py' and the models. You should save this content to a file named 'requirements.txt'.

```
1 requirements_txt_content = '''
2 pandas
3 numpy
4 scikit-learn
5 xgboost
6 shap
7 tensorflow
8 matplotlib
9 seaborn
10 tqdm # For notebook progress bars, maybe not essential for app
11 python-docx # If you need to process docx files in app
12 streamlit # For the example app
13 Pillow # For image processing in digit recognition app
14 '''
15
16 # Save this content to a file (e.g., requirements.txt) manually
17 # with open('requirements.txt', 'w') as f:
18 #     f.write(requirements_txt_content)
19 # print("Content for requirements.txt has been generated.")
20
21 # For demonstration purposes, print the content
22 print("--- Content for requirements.txt ---")
23 print(requirements_txt_content)
24 print("\n--- End of requirements.txt Content ---")
```

Listing 19: Code Implementation

Output

```
--- Content for requirements.txt ---

pandas
numpy
scikit-learn
xgboost
shap
tensorflow
matplotlib
seaborn
tqdm # For notebook progress bars, maybe not essential for app
python-docx # If you need to process docx files in app
streamlit # For the example app
Pillow # For image processing in digit recognition app

--- End of requirements.txt Content ---
```

```
1 app_py_content = '''
2 import streamlit as st
3 import pandas as pd
4 import numpy as np
5 import joblib
6 import tensorflow as tf
7 from tensorflow.keras.models import load_model
8 import shap
9 import matplotlib.pyplot as plt
```

```

10 import seaborn as sns
11 import io
12
13 # Load models
14 xgb_model = joblib.load('xgb_model.joblib')
15 mlp_model = load_model('mlp_model.h5')
16 cnn_model = load_model('cnn_mnist_model.h5')
17
18 # Load X_train for SHAP explainer (or use representative data)
19 X = pd.read_csv('processed.cleveland.data', header=None, names=[
20     'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
21     'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
22 ])
23 X.replace('?', np.nan, inplace=True)
24 for col in X.columns:
25     X[col] = pd.to_numeric(X[col])
26 X.fillna(X.median(), inplace=True)
27 X_train_shap = X.drop('target', axis=1)
28
29 # SHAP explainer for XGBoost
30 xgb_explainer = shap.TreeExplainer(xgb_model)
31
32 # Feature names for heart disease model
33 heart_disease_features = [
34     'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
35     'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
36 ]
37
38 st.set_page_config(layout='wide')
39 st.title("CardioAI Labs Decision Support Tool")
40
41 st.sidebar.header("Choose Module")
42 module_choice = st.sidebar.radio(
43     "Select a task:",
44     ('Heart Disease Prediction', 'Digit Recognition')
45 )
46
47 if module_choice == 'Heart Disease Prediction':
48     st.header("Heart Disease Prediction")
49     st.write("Enter patient data to predict heart disease risk.")
50
51     with st.form("heart_disease_form"):
52         age = st.number_input('Age', min_value=1, max_value=120, value=50)
53         sex = st.selectbox('Sex', options=[(0, 'Female'), (1, 'Male')],
54             format_func=lambda x: x[1])[0]
55         cp = st.selectbox('Chest Pain Type (CP)', options=[(1, 'Typical Angina'), (2, 'Atypical Angina'), (3, 'Non-Anginal Pain'), (4, 'Asymptomatic')],
56             format_func=lambda x: x[1])[0]
57         trestbps = st.number_input('Resting Blood Pressure (trestbps)',
58             min_value=80, max_value=200, value=120)
59         chol = st.number_input('Cholesterol (chol)', min_value=100, max_value=600, value=200)
60         fbs = st.selectbox('Fasting Blood Sugar > 120 mg/dl (fbs)', options=[(0, 'No'), (1, 'Yes')], format_func=lambda x: x[1])[0]
61         restecg = st.selectbox('Resting Electrocardiographic Results (restecg)',
62             options=[(0, 'Normal'), (1, 'ST-T wave abnormality'), (2, 'Left Ventricular Hypertrophy')], format_func=lambda x: x[1])[0]
63         thalach = st.number_input('Maximum Heart Rate Achieved (thalach)',
64             min_value=60, max_value=220, value=150)
65         exang = st.selectbox('Exercise Induced Angina (exang)', options=[(0, 'No'), (1, 'Yes')], format_func=lambda x: x[1])[0]
66         oldpeak = st.number_input('ST Depression Induced by Exercise Relative to Rest (oldpeak)', min_value=0.0, max_value=6.0, value=1.0, step=0.1)

```



```

62     slope = st.selectbox('Slope of the Peak Exercise ST Segment (slope)',
options=[(1, 'Upsloping'), (2, 'Flat'), (3, 'Downsloping')], format_func=
lambda x: x[1])[0]
63     ca = st.selectbox('Number of Major Vessels Colored by Flouroscopy (ca)
', options=[(0, '0'), (1, '1'), (2, '2'), (3, '3')], format_func=lambda x:
str(x[1]))[0]
64     thal = st.selectbox('Thallium Stress Test Result (thal)', options=[(3,
'Normal'), (6, 'Fixed Defect'), (7, 'Reversible Defect')], format_func=
lambda x: x[1])[0]
65
66     predict_button = st.form_submit_button("Predict Heart Disease")
67
68     if predict_button:
69         input_data = pd.DataFrame([[
70             age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak
, slope, ca, thal
71         ]], columns=heart_disease_features)
72
73         # XGBoost Prediction
74         xgb_proba = xgb_model.predict_proba(input_data)[: , 1][0]
75         xgb_pred = (xgb_proba > 0.5).astype(int)
76
77         # Display prediction with color-coded risk label
78         risk_label = "Disease Present" if xgb_pred == 1 else "No Disease"
79         risk_color = "red" if xgb_pred == 1 else "green"
80         st.subheader(f"Prediction: :{risk_color}{[risk_label]}")
81
82         # Model's confidence score
83         st.write(f"Confidence: **{xgb_proba*100:.2f}%**")
84
85         # SHAP explanation for top 3 features
86         shap_values_instance = xgb_explainer.shap_values(input_data)
87         feature_impact = pd.Series(shap_values_instance[0], index=
heart_disease_features)
88         top_3_features = feature_impact.abs().sort_values(ascending=False).head
(3)
89
90         st.write("**Top 3 Features Driving Prediction:**")
91         fig, ax = plt.subplots(figsize=(8, 4))
92         top_3_features.plot(kind='barh', ax=ax, color='skyblue')
93         ax.set_title('Feature Importance')
94         ax.set_xlabel('SHAP Value (Magnitude)')
95         st.pyplot(fig)
96
97         # Plain-English explanation
98         st.write("**Explanation for Nurse:**")
99         explanation_text = "Based on the patient's data, "
100         # Add more specific explanations based on feature_impact, e.g.,
101         # if 'cp' is high and positive impact: "chest pain type was a strong
indicator..."
102         # This part requires more advanced logic to generate dynamically
103         st.write("This patient's health indicators have been analyzed by the
model. The primary factors influencing this prediction are [Feature 1], [
Feature 2], and [Feature 3]. Please review these with the patient.")
104
105
106 elif module_choice == 'Digit Recognition':
107     st.header("Digit Recognition")
108     st.write("Upload an image of a handwritten digit (28x28 grayscale) for
prediction.")
109
110     uploaded_file = st.file_uploader("Choose a digit image...", type=["png", "
jpg", "jpeg"])

```

```
111
112     if uploaded_file is not None:
113         from PIL import Image
114         image = Image.open(uploaded_file).convert('L') # Convert to grayscale
115         image = image.resize((28, 28))
116         img_array = np.array(image) / 255.0
117         img_array = img_array.reshape(1, 28, 28, 1)
118
119         st.image(image, caption='Uploaded Image', use_column_width=True)
120         st.write("")
121
122         if st.button("Predict Digit"):
123             predictions = cnn_model.predict(img_array)
124             predicted_digit = np.argmax(predictions)
125             confidence = np.max(predictions) * 100
126
127             st.subheader(f"Predicted Digit: **{predicted_digit}**")
128             st.write(f"Confidence: **{confidence:.2f}%**")
129     '''
130
131 # Save this content to a file (e.g., app.py) manually or programmatically
132     outside the notebook
133 # with open('app.py', 'w') as f:
134 #     f.write(app_py_content)
135 # print("Content for app.py has been generated.")
136
137 # For demonstration purposes, print the content
138 print("--- Content for app.py ---")
139 print(app_py_content)
140 print("\n--- End of app.py Content ---")
```

Listing 20: Code Implementation

Output

```
--- Content for app.py ---
```

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
import tensorflow as tf
from tensorflow.keras.models import load_model
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import io

# Load models
xgb_model = joblib.load('xgb_model.joblib')
mlp_model = load_model('mlp_model.h5')
cnn_model = load_model('cnn_mnist_model.h5')

# Load X_train for SHAP explainer (or use representative data)
X = pd.read_csv('processed.cleveland.data', header=None, names=[
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
])
X.replace('?', np.nan, inplace=True)
for col in X.columns:
    X[col] = pd.to_numeric(X[col])
X.fillna(X.median(), inplace=True)
X_train_shap = X.drop('target', axis=1)

# SHAP explainer for XGBoost
xgb_explainer = shap.TreeExplainer(xgb_model)

# Feature names for heart disease model
heart_disease_features = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
]

st.set_page_config(layout='wide')
st.title("CardioAI Labs Decision Support Tool")

st.sidebar.header("Choose Module")
module_choice = st.sidebar.radio(
    "Select a task:",
    ('Heart Disease Prediction', 'Digit Recognition')
)

if module_choice == 'Heart Disease Prediction':
    st.header("Heart Disease Prediction")
    st.write("Enter patient data to predict heart disease risk.")

    with st.form("heart_disease_form"):
        age = st.number_input('Age', min_value=1, max_value=120, value=50)
        sex = st.selectbox('Sex', options=[(0, 'Female'), (1, 'Male')], format_func=lambda x: x[1])
        cp = st.selectbox('Chest Pain Type (CP)', options=[(1, 'Typical Angina'), (2, 'Atypical A
        trestbps = st.number_input('Resting Blood Pressure (trestbps)', min_value=80, max_value=2
        chol = st.number_input('Cholesterol (chol)', min_value=100, max_value=600, value=200)
        fbs = st.selectbox('Fasting Blood Sugar > 120 mg/dl (fbs)', options=[(0, 'No'), (1, 'Yes')
        restecg = st.selectbox('Resting Electrocardiographic Results (restecg)', options=[(0, 'No
        thalach = st.number_input('Maximum Heart Rate Achieved (thalach)', min_value=60, max_valu
        exang = st.selectbox('Exercise Induced Angina (exang)', options=[(0, 'No'), (1, 'Yes')],
        oldpeak = st.number_input('ST Depression Induced by Exercise Relative to Rest (oldpeak)',
        slope = st.selectbox('Slope of the Peak Exercise ST Segment (slope)', options=[(1, 'Upward
```

```
1 requirements_txt_content = '''
2 pandas
3 numpy
4 scikit-learn
5 xgboost
6 shap
7 tensorflow
8 matplotlib
9 seaborn
10 tqdm # For notebook progress bars, maybe not essential for app
11 python-docx # If you need to process docx files in app
12 streamlit # For the example app
13 Pillow # For image processing in digit recognition app
14 '''
15
16 # Save this content to a file (e.g., requirements.txt) manually
17 # with open('requirements.txt', 'w') as f:
18 #     f.write(requirements_txt_content)
19 # print("Content for requirements.txt has been generated.")
20
21 # For demonstration purposes, print the content
22 print("--- Content for requirements.txt ---")
23 print(requirements_txt_content)
24 print("\n--- End of requirements.txt Content ---")
```

Listing 21: Code Implementation

Output

```
--- Content for requirements.txt ---

pandas
numpy
scikit-learn
xgboost
shap
tensorflow
matplotlib
seaborn
tqdm # For notebook progress bars, maybe not essential for app
python-docx # If you need to process docx files in app
streamlit # For the example app
Pillow # For image processing in digit recognition app

--- End of requirements.txt Content ---
```